1. Servicios Web REST/APIs REST

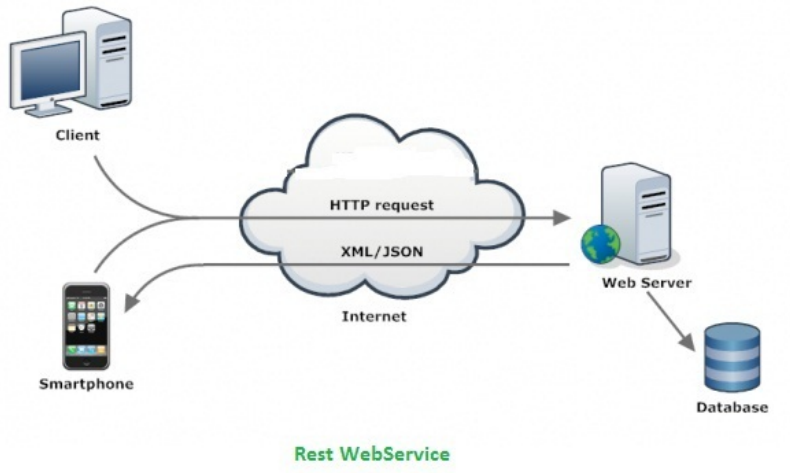
Tabla de contenidos

1.1. Reglas de diseño REST 2
1.2. Uso correcto de verbos y código de estado HTTP 2
1.3. Uso de sustantivos en puntos finales y nombres en plural 3
1.4. Uso correcto de filtrado, clasificación y paginación. 3
1.5. APIs REST con Spring Boot 4
1.5.1. Anotaciones REST 4
1.5.2. Procesamiento Datos de Entrada 6
1.5.3. Parámetros incrustados en la URL de petición (@PathVariable) 6
1.5.4. Parametros de Petición (@RequestParam) 7
1.5.5. Datos de Petición POST/PUT (@RequestBody) 7
1.5.6. Validación de Peticiones (Bean Validator) 9
1.6. Manejo de Errores y Excepciones 10
1.6.1. @ResponseStatus 11
1.6.2. Customización de la Respuesta de Error 11
1.6.3. @ExceptionHandler 12
1.6.4. @ControllerAdvice 13
1.6.5. ResponseEntityExceptionHandler 14
1.6.6. Cómo procesa Spring las excepciones 15

Un servicio web es una aplicación que se encuentra en el lado servidor y permite que otra aplicación cliente conecte con ella a través de Internet para el intercambio de información utilizando el protocolo HTTP.

Una de las principales características de los servicios web es que no es necesario que ambas aplicaciones (servidor y cliente) estén escritas en el mismo, lo que hace que la interoperabilidad sea máxima. Por ejemplo, podríamos crear un servicio web en Python y utilizarlo conectándonos desde una aplicación móvil con Android, desde otra aplicación programada en Java o incluso desde otro servicio web escrito con .NET.

Además, utilizan el protocolo HTTP para el intercambio de información, lo que significa que la conexión se establece por el puerto 80 que es el mismo que utilizan los navegadores y que es prácticamente seguro que se encuentre abierto en cualquier organización protegida por firewall. Esto hace que no sea necesario tener especial cuidado abriendo puertos innecesarios para poder conectarnos a ellos. Antes de la llegada de los servicios web como los conocemos ahora existían otros protocolos más complicados que requerían de servicios y puertos adicionales, incrementando el riesgo de ataques en las organizaciones que los ponían en marcha.



Los Servicios Web REST son Servicios Web que cumplen una serie de requisitos según un patrón de arquitectura definida hacia el año 2000 y que se ha extendido siendo el patrón predominante a la hora de implementar este tipo de aplicaciones.

Básicamente consiste en seguir una serie de reglas que definen dicha arquitectura. Entre ellas están el uso del procotolo HTTP por ser el más extendido a lo largo de Internet en la actualidad. Además, cada recurso del servicio web tiene que ser identificado por una dirección web (una URL) siguiendo una estructura determinada. Además, la respuesta tendrá que tener una estructura determinada en forma de texto que normalmente vendrá en alguno de los formatos abiertos más conocidos como XML o JSON.

1.1. Reglas de diseño REST

A continuación se describen las reglas principales de construcción de APIs siguiendo los principios y comportamiento definidos en los libros **REST API Design Rulebook** y **RESTful web API Handbook.**

Las reglas principales del diseño son:

1. Uso correcto de verbos y código de estado HTTP.
2. Uso de sustantivos en puntos finales y nombres en plural.
3. Uso correcto de filtrado, clasificación y paginación.

1.2. Uso correcto de verbos y código de estado HTTP

Las pautas de REST sugieren usar un verbo HTTP específico dependiendo de la particularidad de la llamada. REST debe respetar tanto los verbos como los códigos de estado para cada operación.

A continuación los verbos empleados correctamente dependiendo de la acción a realizar en el servidor.

- **GET:** Solicita información de recursos.
- **POST:** Creación de nuevos recursos.

- **PUT:** Actualiza un recurso existente en su totalidad.
- **PATCH:** Actualiza un recurso existente parcialmente.
- **DELETE:** Elimina un recurso existente.

A continuación los códigos de estado utilizados correctamente dependiendo de la situación.

- **1XX:** Respuestas informativas.
- **2XX:** Peticiones correctas.
- **3XX:** Redirecciones.
- **4XX:** Errores del cliente.
- **5XX:** Errores del servidor.

A continuación los estados más utilizados y una breve descripción:

- **200 (OK):** La solicitud ha sido recibida, entendida y procesada correctamente.
- **201 (Created):** La solicitud se procesó correctamente y generó un nuevo recurso en el proceso.
- **400 (Bad Request):** Solicitud no apropiada, faltan requisitos para ser válida.
- **401 (Unauthorized):** No autorizado credenciales inválidas.
- **403 (Forbidden):** Prohibido credenciales insuficientes, no tiene permisos para acceder.
- **404 (Not found):** Recurso no encontrado.
- **500 (Internal server error):** Hubo un error en el servidor y la solicitud no pudo ser completada.

1.3. Uso de sustantivos en puntos finales y nombres en plural

En el correcto diseño de una API del estilo REST, se considera utilizar sustantivos y nombres en plural en nuestros puntos finales y **NO** verbos de acción, esto es básicamente porqué la acción ya está determinada en los verbos de HTTP.

Incorrecto #

```
POST https://api.com/v1/createUser
```

Correcto ✓

```
POST https://api.com/v1/users
```

1.4. Uso correcto de filtrado, clasificación y paginación.

Para realizar las operaciones de filtrado, clasificación y paginación en nuestra API REST, lo debemos hacer mediante *Query Strings*. Las *Query Strings* o cadenas de consultas son un

término que se utiliza para hacer referencia a una interacción adicional sobre un recurso o base de datos.

A continuación algunas referencias y estandarizaciones para algunas acciones:

- **Filtrado de recursos**

```
GET https://api.com/v1/users?first_name=diego&last_name=cortes
```

- **Orden de recursos**

```
GET https://api.com/v1/users?sort=username
```

- **Paginación de recursos**

```
GET https://api.com/v1/users?page=0&size=20
```

- **Buscador de resultados**

```
GET https://api.com/v1/users?username=diego
```

1.5. APIs REST con Spring Boot

La implementación de las APIs REST en Spring Boot se basan en el uso del **Controlador**. **Un controlador va a ser el punto de entrada de nuestra aplicación** y su función será interceptar la llamada, adaptar los datos de entrada, procesarlos y devolverlos.

La defición de los controladores en Spring Boot se realiza usando un conjunto específico de anotaciones.

1.5.1. Anotaciones REST

Spring Boot cuena con un gran número de anotaciones, cada una de las cuales tiene una función específica. Resulta complicado estudiar cada una de ellas en profundidad, sin embargo, a continuación se detallan las más habituales cuando se trabaja con APIs REST:

1. **@Controller:** Se utiliza en clases Java que desempeñan el papel de controlador en su aplicación. Permite la autodetección de clases de componentes en el classpath y las definiciones de bean de registro automático para ellas. Para habilitar la detección automática de dichos controladores anotados, puede agregar el escaneo de componentes a su configuración. La clase Java anotada con **@Controller** es capaz de manejar múltiples asignaciones de solicitudes.

2. **@RequestMapping:** Se usa tanto a nivel de clase como de método. Se utiliza para asignar solicitudes web a clases de manejador y métodos de manejador específicos. Cuando **@RequestMapping** se usa en el nivel de clase, crea un URI base para el que se usará el controlador. Cuando esta anotación se utiliza en los métodos, le dará el URI en el que se ejecutarán los métodos del controlador. A partir de esto, puede inferir que la asignación de solicitud a nivel de clase seguirá siendo la misma, mientras que cada método de controlador tendrá su propia asignación de solicitud.

A veces es posible que desee realizar diferentes operaciones en función del método HTTP utilizado, aunque el URI de la solicitud puede permanecer igual. En tales situaciones, puede usar el atributo de método de **@RequestMapping** con un valor de método HTTP para limitar los métodos HTTP para invocar los métodos de su clase. Por ejemplo: *@RequestMapping(value = "/personas", method = RequestMethod.GET)*.

Spring Framework 4.3 introdujo las siguientes variantes de nivel de método de la anotación **@RequestMapping** para expresar mejor la semántica de los métodos anotados. El uso de estas anotaciones se ha convertido en el método estándar para definir los puntos finales. Actúan como envoltorios para **@RequestMapping**.

a. **@GetMapping:** Se utiliza para asignar solicitudes HTTP GET a métodos de controlador específicos. **@GetMapping** es una anotación compuesta que actúa como un acceso directo para **@RequestMapping** (method = RequestMethod.GET).

b. **@PostMapping:** Se utiliza para asignar solicitudes HTTP POST a métodos de controlador específicos. **@PostMapping** es una anotación compuesta que actúa como un acceso directo para **@RequestMapping** (method = RequestMethod.POST)

c. **@PutMapping:** Se utiliza para mapear solicitudes HTTP PUT en métodos de manejador específicos. **@PutMapping** es una anotación compuesta que actúa como un acceso directo para **@RequestMapping** (method = RequestMethod.PUT).

d. **@PatchMapping:** Se usa para mapear solicitudes HTTP PATCH en métodos de manejador específicos. **@PatchMapping** es una anotación compuesta que actúa como un acceso directo para **@RequestMapping** (method = RequestMethod.PATCH).

e. **@DeleteMapping:** Se usa para asignar solicitudes HTTP DELETE a métodos de controlador específicos. **@DeleteMapping** es una anotación compuesta que actúa como un acceso directo para **@RequestMapping** (method = RequestMethod.DELETE).

3. **@PathVariable:** Se utiliza para anotar argumentos del método del controlador de solicitudes. La anotación **@RequestMapping** se puede usar para manejar cambios dinámicos en el URI donde un cierto valor de URI actúa como parámetro. Puede especificar este parámetro usando una expresión regular. La anotación **@PathVariable** se puede usar para declarar este parámetro (Ejemplo: {id}).

4. **@PathParam:** Se utiliza para anotar argumentos del método del controlador de solitudes de forma similar a **@PathVariable**. Dicha anotación nos permite enlazar el parámetro anotado con un parámetro recibido en la URL de petición. Se utiliza en las peticiones de tipo GET.

5. **@RequestParam:** Se utiliza para anotar argumentos del método del controlador de solitudes de forma similar a **@PathParam**. Dicha anotación nos permite enlazar el parámetro anotado con un parámetro recibido en la URL de petición. Se utiliza en las peticiones de tipo GET.

6. **@RequestBody:** Se utiliza para anotar argumentos del método del controlador de solicitudes. La anotación **@RequestBody** indica que un parámetro de método debe estar vinculado al valor del cuerpo de la solicitud HTTP. El *HttpMessageConveter* es responsable de convertir del mensaje de solicitud HTTP a objeto.

7. **@ResponseBody:** Se utiliza para anotar métodos de manejo de solicitudes. La anotación **@ResponseBody** es similar a la anotación **@RequestBody**. La anotación **@ResponseBody** indica que el tipo de resultado debe escribirse directamente en el cuerpo de la respuesta en cualquier formato que especifique, como JSON o XML. Spring convierte el objeto devuelto en un cuerpo de respuesta mediante el *HttpMessageConveter*.

8. **@RestController:** Se usa a nivel de clase. La anotación **@RestController** marca la clase como un controlador donde cada método devuelve un objeto de dominio en lugar de una vista. Al anotar una clase con esta anotación, ya no necesita agregar **@ResponseBody** a todos los métodos *RequestMapping*. Significa que ya no usa resuelve vistas ni envía HTML en respuesta. Simplemente envía el objeto de dominio como una respuesta HTTP en el formato que entienden los consumidores, como JSON. **@RestController** actua como una unión de **@Controller** y **@ResponseBody**.

En base a dichas anotaciones un controlador tipo quedaría tal que así:

```
@RestController
public class ApplicationController {

 @GetMapping("/personas")
 public ResponseEntity<String> busquedaPersonas(@PathParam("nombre") String nombre,
   @PathParam("apellido") String apellido) {
  return ResponseEntity.ok().body(nombre + " " + apellido);
 }

 @PostMapping("/personas")
 public ResponseEntity<String> busquedaPersonas(@RequestBody Map<String, String> parametros) {
  return ResponseEntity.ok().body(parametros.get("nombre") + " " + parametros.get("apellido"));
 }
}
```

1.5.2. Procesamiento Datos de Entrada

A la hora de pasar datos en una petición a una API Rest se pueden usar distintas técnicas, todas ellas asociadas a peticiones HTTP específicas. Debido a los distintos escenarios que se pueden presentar, Spring Boot incluye distintos mecanismos para recuperar los datos de la petición en base a la ténica usada para realizar la petición.

A continuación vamos a desarrollar algunas de las posibilidades.

1.5.3. Parámetros incrustados en la URL de petición (@PathVariable)

La anotación **@PathVariable** es una anotación que nos sirve dentro de Spring Boot para configurar variables dentro de los propios segmentos de la URL, algo que a nivel de Arquitecturas REST es imprescindible.

Veamos un ejemplo de URL donde se incrustan dos variables:

```
http://localhost:9999/personas/{idPersona}/direcciones/{idDireccion}
```

En esta URL tenemos dos variables (idPersona y idDireccion) cada una de las cuales va a contener el identificador de persona y dirección respectivamente.

A través de la anotación **@PathVariable** podemos enlazar dichos valores con sendos parámetros del método controlador. La forma de hacerlo será la siguiente:

```
@GetMapping("/personas/{idPersona}/direcciones/{idDireccion}")
```

```
public ResponseEntity<String> ejemplo(@PathVariable("idPersona") Long idPersona,
 @PathVariable("idDireccion") Long idDireccion) {
 return ResponseEntity.ok().body(idPersona + " " + idDireccion);
}
```

La anotación **@PathVariable** admite varios atributos:

1. **value:** pemite especificar el nombre de la variable que se incluye en la URL. Este atributo se usa cuando queremos usar un nombre distinto para la variable y el parámetro del método controlador. **@PathVariable("id") y @PathVariable(value="id") son equivalentes**.

   Si no se especifica el atributo value el nombre del parámetro de método debe coincidir con el nombre de la variable de URL.

2. **required:** permite especificar si una variable es opcional. Por defecto todas las PathVariable son obligatorias, en caso de que sea opcional es necesario especificarlo indicando el atributo **required = false**. En caso de que especifiquemos una PathVariable como opcional es necesario indicar ambas rutas (con y sin la variable) en el mapeo de petición.

```
@GetMapping(value = { "/personas", "/personas/{id}" })
public ResponseEntity<String> ejemplo(@PathVariable(value="id", required=false) Long idPersona)
```

1.5.4. Parametros de Petición (@RequestParam)

La anotación **@RequestParam** se utiliza para indicarle a Spring Boot en que parámetro del método controlador se debe de inyectar el valor de los parámetros que se reciben en la petición.

A continuación se muestra un ejemplo de uso:

```
http://localhost:9999/personas?nombre=Manuel&apellido=Pérez
```

```
@GetMapping("/personas")
public ResponseEntity<String> ejemplo(@RequestParam("nombre") String nombre,
                                      @RequestParam("apellido") String apellido)
```

La anotación **@RequestParam** admite varios atributos:

1. **value:** pemite especificar el nombre de la variable que se incluye en la URL. Este atributo se usa cuando queremos usar un nombre distinto para el parámetro de la petición y el parámetro del método controlador. **@RequestParam("id") y @RequestParam(value="id") son equivalentes**.

   Si no se especifica el atributo value el nombre del parámetro de método debe coincidir con el nombre de la variable de URL.

2. **required:** permite especificar si un parámetro es opcional. Por defecto todas los RequestParam son obligatorios, en caso de que sea opcional es necesario especificarlo indicando el atributo **required = false**.

1.5.5. Datos de Petición POST/PUT (@RequestBody)

El método **POST** introduce los parámetros **en la solicitud HTTP** de forma que no quedan visibles para el usuario. Los datos se encapsulan en el cuerpo de la solicitud por lo tanto, la forma

de recuperar esa información desde un controlador diferirá de los mecanismos anteriormente descritos.

El procesamiento de las peticiones **POST** en Spring Boot está intimamente ligado al uso de la anotación **@RequestBody**. La anotación **@RequestBody** se utiliza para indicarle a Spring Boot que el cuerpo de la solicitud debe ser leído y deserializado en un **Object**.

En resumen, esta anotación indica que un parámetro del método controlador debe vincularse al cuerpo de la solicitud web, el cuerpo de la solicitud es pasado por un **HttpMessageConverter** para resolver el argumento del método, Spring Boot detecta el tipo de contenido de la solicitud y utiliza el conversor adecuado para su parseo.

En nuestro caso, todas las peticiones POST y PUT usarán contenido de tipo JSON.

Consideremos una petición POST (*http://localhost:9999/usuarios*) con el siguiente cuerpo de petición JSON.

```
{
    "nombre":"Antonio",
    "apellido":"Primavera",
}
```

Vamos a ver como sería el controlador que procesaría dicha petición.

```
@PostMapping("/usuarios")
public ResponseEntity<String> ejemplo(@RequestBody Usuario usuario)
```

Para que el ejemplo esté completo necesitamos definir la clase Usuario que será la clase en la que se parserán los datos de entrada.

```
public class Usuario {

    private String nombre;
    private String apellido;

    public void setNombre(String nombre){
        this.nombre=nombre;
    }

    public String getNombre(){
        return this.nombre;
    }

    public void setApellido(String apellido){
        this.apellido=apellido;
    }

    public String getApellido(){
        return this.apellido;
    }

}
```

El procedimiento es simple, al recibir la petición se recupera el "body" o cuerpo de la solicitud y se crea una nueva instancia de la clase Usuario asignado el valor de cada uno de los atributos. **Para que este mecanismo funcione correctamente es indispensable que los atributos tengan el mismo nombre que los recibidos en el JSON de entrada de la petición**.

Cuando el JSON de petición es plano (sólo duplas nombre:valor) también tenemos la posibilidad de realizar el parseo a un mapa de valores.

```
@PostMapping("/usuarios")
```

```
public ResponseEntity<String> ejemplo(@RequestBody Map<String, String> parametros)
```

La anotación **@RequestBody** admite el atributo **required** que permite especificar si el parámetro es opcional. Por defecto el valor de este atributo es true, en caso de que el parámetro sea opcional es necesario especificarlo indicando el atributo **required = false**

1.5.6. Validación de Peticiones (Bean Validator)

Spring boot proporciona diferentes formas de validar los request, que van desde mecanismos simples, hasta complejas estructuras de clases de validación. Las funcionalidades de validación se incluyen a través de la siguiente dependencia:

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

La primera forma es usando anotaciones de validación en los parámetros del método del controlador. Por ejemplo, si quieres validar que un parámetro de tipo String no sea nulo ni vacío, puedes usar la anotación @NotNull y @Size.

```
@GetMapping("/api/usuarios")
public ResponseEntity<String> example(@NotNull @Size(min = 5) @RequestParam String nombre) {
    // Tu código aquí
}
```

Este mecanismo es muy útil para la validación de los parámetros de las peticiones GET, pero quizá el escenario en el que resulta más interesante la validación de datos es en los procesos de creación/modificación de elementos mediante peticiones POST/PUT. En estos casos es habitual usar un objeto para el parseo de los datos mediante la anotación **@RequestBody**, en cuyo caso el proceso de validación se basa en el uso de la anotación **@Valid**.

La anotación **@Valid** ayuda a que Spring Boot valide el request antes de ejecutar el método, lo cual es de gran ayuda a evitar que se ejecute el método en caso de los datos no sean correctos.

El proceso es muy simple, se anota el parámetro del médodo de controlador con la anotación @Valid, y adicionalmete anotamos los atributos de la clase en la que serán parseados los datos con las restricciones a aplicacar a cada uno de ellos.

Bean Validation proporciona un conjunto de anotaciones que puedes usar para validar los atributos de una clase. Algunas de las anotaciones más comunes son:

- **@NotNull:** indica que el atributo no puede ser nulo.
- **@Size:** indica el tamaño mínimo y/o máximo permitido para un atributo de tipo String o Collection.
- **@Min** y **@Max:** indican el valor mínimo y/o máximo permitido para un atributo numérico.
- **@Email:** indica que el atributo debe ser una dirección de correo electrónico válida.
- **@Past** y **@Future:** indican que un atributo de tipo Date o Calendar debe ser una fecha en el pasado o en el futuro, respectivamente.
- **@Pattern:** indica un patrón que debe cumplir un atributo de tipo String.

- **@DecimalMin** y **@DecimalMax:** indican el valor mínimo y/o máximo permitido para un atributo de tipo BigDecimal.
- **@NotEmpty:** indica que un atributo de tipo String, Collection o Map no puede ser vacío.

A continuación podemos observar un ejemplo de clase anotada para su validación.

```
public class Libro {

    @NotNull
    @NotEmpty
    private String titulo;

    @DecimalMin("0.00")
    @DecimalMax("9999.99")
    private BigDecimal precioMinimo;

    @DecimalMin("0.00")
    @DecimalMax("9999.99")
    private BigDecimal precioMaximo;

    @Past
    private Date fechaPublicacion;

}
```

La invocación a la validación se realiza de la siguiente forma:

```
public ResponseEntity<String> crearLibro(@RequestBody @Valid Libro libro)
```

Para que el procedimiento de validación se realice es necesario anotar la clase de controlador con @Validate.

```
@RestController
@Validated
public class ApplicationController {
```

1.6. Manejo de Errores y Excepciones

Spring Boot nos proporciona herramientas para gestionar excepciones más allá de simples bloques «try-catch». Para usar estas herramientas, usamos varias anotaciones que nos permiten manejar la gestión de excepciones como un elemento transversal:

- **@ResponseStatus**
- **@ExceptionHandler**
- **@ControllerAdvise**

Antes de adentrarnos en estas anotaciones revisaremos cómo gestiona Spring las excepciones lanzadas por nuestros controladores web – nuestra última línea defensiva para capturar una excepción.

Por defecto, cualquier excepción de servidor genera una respuesta de error con estado HTTP 500. El aspecto de la respuesta generada es el siguiente:

```
{
  "timestamp": "2024-01-22T11:16:42.050+00:00",
  "status": 500,
  "error": "Internal Server Error",
  "path": "/api-daw/usuarios"
}
```

Sin embargo, en la mayoría de los casos esta respuesta no es lo suficientemente descriptiva y podemos encontrarnos con situaciones en las que es necesario generar una respuesta mucho más concreta. El primer paso para conseguir errores más específicos es hacer uso de Excepciones específicas.

```
public class UserNotFoundException extends RuntimeException {

 private static final long serialVersionUID = 43876691117560211L;

 public UserNotFoundException(Long id) {
  super("No se puede encontrar el usuario con la ID: " + id);
 }

}
```

Una vez generada/as las excepciones espefíficas, Spring Boot cuenta con diferentes soluciones para su menejo, todas ellas subceptibles de ser usadas según el escenario en el que nos encontremos.

1.6.1. @ResponseStatus

@ResponseStatus nos permite modificar el estado HTTP de nuestra respuesta. Se puede usar en los siguientes lugares:

- En la propia clase de excepción.
- En métodos, junto con la anotación **@ExceptionHandler**.
- En clases, junto con la anotación **@ControllerAdvice**.

Veamos la forma más simple de usarlo:

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class UserNotFoundException extends RuntimeException {

    private static final long serialVersionUID = 43876691117560211L;

    public UserNotFoundException(Long id) {
        super("No se puede encontrar el usuario con la ID: " + id);
    }

}
```

Una vez anotada la excepción, cada vez que se produzca el error se mostrará una nueva respuesta.

```
{
  "timestamp": "2024-01-22T11:31:11.279+00:00",
  "status": 404,
  "error": "Not Found",
  "path": "/api-daw/usuarios"
}
```

1.6.2. Customización de la Respuesta de Error

Por defecto, la respuesta que genera Spring Boot ante una excepción contiene los siguientes campos:

- Fecha.
- Estado.

- Error.
- Mensaje.
- Ruta en la que se ha producido.

Sin embargo, hay casos en los que puede resultar necesario modificar dichos campos para complementarlos o para reducirlos. El mecanismo para hacerlo es crear nuestra propia respuesta mediante una clase de error propia.

```
public class ApiError {

    private HttpStatus estado;
    @JsonFormat(shape = Shape.STRING, pattern = "dd/MM/yyyy hh:mm:ss")
    private LocalDateTime fecha;
    private String mensaje;
    ...

}
```

En el caso anterior, nos hemos creado nuestra propia clase que contendrá sólo tres atributos, estado, fecha y mensaje. Veamos cómo emplear nuestra propia clase para la generación de la respuesta.

1.6.3. @ExceptionHandler

La anotación **@ExceptionHandler** nos da mucha flexibilidad en términos de gestión de excepciones. Acepta como parámetro una o varias excepciones:

- @ExceptionHandler(TheException.class)
- @ExceptionHandler({OneException.class, TwoException.class})

Spring Boot nos permite ubicarla en un método de cualquier controlador, dicho método puede recibir un gran número de parámetros (según nuestra necesidad):

- Una instancia de una excepción.
- Objetos Request, Response.
- Objetos de sesión (HttpSession).
- ...

También podrá tener distintos tipos de respuesta:

- void.
- Model, ModelAndView, Map (motor de plantillas).
- String.
- @ResponseBody.
- HttpEntity, ResponseEntity.

Vamos a ver un ejemplo:

```
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<ApiError> handleUsuarioNoEncontrado(UserNotFoundException ex) {
    ApiError apiError = new ApiError();
    apiError.setEstado(HttpStatus.NOT_FOUND);
    apiError.setFecha(LocalDateTime.now());
    apiError.setMensaje(ex.getMessage());
    return ResponseEntity.status(HttpStatus.NOT_FOUND).body(apiError);
}

@ExceptionHandler(JsonMappingException.class)
public ResponseEntity<ApiError> handleJsonMappingException(JsonMappingException ex) {
    ApiError apiError = new ApiError();
    apiError.setEstado(HttpStatus.BAD_REQUEST);
    apiError.setFecha(LocalDateTime.now());
    apiError.setMensaje(ex.getMessage());
    return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(apiError);
}
```

Como podemos observar, hemos creado dos métodos.

El primero de ello destinado al procesamiento de las excepciones de tipo *UserNotFoundException*, la respuesta se genera a partir de la case de error generada en el apartado anterior.

El segundo método es muy útil y de uso general, se trata de un método para el manejo de los errores de parseo de valores al manipular los datos de la aplicación (*JsonMappingException*).

La anotación **@ExceptionHandler** se ubica a nivel de método y funciona a nivel de clase, esto nos obliga a tener que hacer un tratamiento de excepciones específico para cada clase de controladores de nuestra aplicación. Adicionalmente, para aquellos tipos de excepciones que se lanzan en distintas clases tendremos que duplicar código.

Este problema debe solventarse mediante un manejo de excepciones de ámbito global.

1.6.4. @ControllerAdvice

El término ‘Advise’ viene de la Programación Orientada a Aspectos (Aspect-Oriented Programming, AOP) que nos permite inyectar código transversal (llamado «advise») en métodos existentes. Un advise de controlador nos permite interceptar y modificar los valores de retorno de un método de controlador, en nuestro caso para gestionar excepciones.

Las clases **@ControllerAdvice** nos permites aplicar la gestión de excepciones a uno, a varios o a todos los controladores de nuestra aplicación. En nuestro caso, y teniendo en cuenta que estamos trabajando con APIs REST, haremos uso de la anotación **@RestControllerAdvice** es una especialización que unifica **@ControllerAdvice** y **@ResponseBody**.

Al anotar una clase con @RestControllerAdvice, le estamos indicando a Spring Boot que en el momento de manejar una excepción, debe ser esta clase la que establezca quién procesa dicha excepción. Se escogerá el primer método dentro de la clase @RestControllerAdvice que esté anotado para tratar la excepción correspondiente (con @ExceptionHandler).

```
@RestControllerAdvice
public class GlobalControllerAdvice {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<ApiError> handleUsuarioNoEncontrado(UserNotFoundException ex) {
        ApiError apiError = new ApiError();
        apiError.setEstado(HttpStatus.NOT_FOUND);
        apiError.setFecha(LocalDateTime.now());
        apiError.setMensaje(ex.getMessage());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(apiError);
    }
```

```
    @ExceptionHandler(JsonMappingException.class)
    public ResponseEntity<ApiError> handleJsonMappingException(JsonMappingException ex) {
        ApiError apiError = new ApiError();
        apiError.setEstado(HttpStatus.BAD_REQUEST);
        apiError.setFecha(LocalDateTime.now());
        apiError.setMensaje(ex.getMessage());
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(apiError);
    }

}
```

Como se observa en el ejemplo, hemos desplazado los métodos de la clase de controladores a la case de control global de excepciones.

Si no indicamos nada, la anotación hace que la clase trate posibles excepciones producidas en cualquier controlador. Sin embargo, podemos acotar el radio de acción:

- *@ControllerAdvice("my.chosen.package")*.
- *@ControllerAdvice(value = "my.chosen.package")*.
- *@ControllerAdvice(basePackages = "my.chosen.package")*.
- *@ControllerAdvice(basePackageClasses = MyClass.class)*.
- *@ControllerAdvice(assignableTypes = MyController.class)*.
- *@ControllerAdvice(annotations = RestController.class)*.

Puede existir más de una clase anotada con **@RestControllerAdvice**. En tal caso, puede ser recomendable el uso de **@Order** o **@Priority** para establecer una predecencia en el tratamiento de errores.

En caso de varias opciones para una excepción dentro de una clase, Spring Boot escogerá la más cercana a la raíz. Por ejemplo, si tenemos un método para el tratamiento de *IOException* y otro para el tratamiento de *FileNotFoundException*, al producirse una *FileNotFoundException* se ejecutará el método de *FileNotFoundException* y no el de *IOException*.

1.6.5. ResponseEntityExceptionHandler

Según la propia documentación de Spring Boot, **ResponseEntityExceptionHandler** se trata de una clase base conveniente para las clases *@ControllerAdvice* que desean proporcionar un manejo centralizado de excepciones en todos los métodos *@RequestMapping* a través de los métodos *@ExceptionHandler*.

ResponseEntityExceptionHandler se proporciona una larga lista de métodos que podemos sobrescribir, entre ellos destacan:

- ***handleExceptionInternal***
- *handleMissingPathVariable*
- *handleTypeMismatch*

Para nosotros, el más importante es el método **handleExceptionInternal**. Según la documentación, dicho método *es un único lugar para personalizar el cuerpo de respuesta de todos los tipos de excepción*. Si lo sobrescribimos, y lo personalizamos

con nuestra clase específica de error, el resto de errores manejados por la clase base *ResponseEntityExceptionHandler* utilizarán nuestra clase de error.

```
@RestControllerAdvice
public class GlobalControllerAdvice extends ResponseEntityExceptionHandler {

    @ExceptionHandler(ProductoNotFoundException.class)
    public ResponseEntity<ApiError> handleProductoNoEncontrado(ProductoNotFoundException ex) {
        ApiError apiError = new ApiError();
        apiError.setEstado(HttpStatus.NOT_FOUND);
        apiError.setFecha(LocalDateTime.now());
        apiError.setMensaje(ex.getMessage());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(apiError);
    }

    @Override
    protected ResponseEntity<Object> handleExceptionInternal(Exception ex, Object body, HttpHeaders
 headers,
  HttpStatus status, WebRequest request) {
        ApiError apiError = new ApiError();
        apiError.setEstado(status);
        apiError.setFecha(LocalDateTime.now());
        apiError.setMensaje(ex.getMessage());
        return ResponseEntity.status(status).headers(headers).body(apiError);
    }

}
```

1.6.6. Cómo procesa Spring las excepciones

Ahora que hemos visto los mecanismos disponibles para gestionar las excepciones en Spring, vamos a intentar entender brevemente cómo las maneja Spring Boot y cuál es el flujo de procesamiento y dónde cobra importancia cada uno de los mecanismos.

En el siguiente diagrama de flujo se muestra el proceso de gestión de excepciones que sigue Spring Boot si no construimos nuestros propios gestores de excepciones:

